Отчёт по лабораторной работе №4

Компьютерный практикум по статистическому анализу

Арбатова Варвара Петровна

В данном отчёте представлены результаты выполнения лабораторной работы по теме ...

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

Содержание

1 Введение **6**
1.1 Актуальность 6
1.2 Цель работы 6
1.3 Задачи 6
1.4 Объект и предмет исследования 7
1.5 Методы исследования 7
1.6 Схема выполнения работы 9

2 Теоретическая часть **10**

3 Материалы и методы **11**

4 Выполнение лабораторной работы **12**
4.1 Примеры 12
4.2 Задания 16

5 Результаты **20**

6 Обсуждение результатов **21**

7 Заключение **22**

Список литературы **23**

A Приложение А. Полный листинг программы **24**

B Приложение Б. Результаты бенчмаркинга **25**

Список иллюстраций

1.1 Блок-схема выполнения лабораторной работы 9

Список таблиц

1 Введение

1.1 Актуальность

Линейная алгебра лежит в основе большинства вычислительных задач: от решения систем линейных уравнений до спектрального анализа и экономического моделирования. Современные языки программирования, такие как Julia, предоставляют эффективные и удобные средства для работы с матрицами и векторами. Освоение этих средств позволяет быстро и надёжно решать прикладные задачи.

1.2 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

1.3 Задачи

- освоить поэлементные операции над многомерными массивами;
- изучить операции транспонирования, вычисления следа, ранга, определителя и обращения матриц;
- научиться вычислять нормы векторов и матриц, углы и расстояния между векторами;

- освоить матричное умножение, скалярное произведение и работу со специальными матричными структурами;
- изучить факторизации (LU, QR) и спектральное разложение;
- выполнить оценку эффективности операций с помощью пакета BenchmarkTools;
- решить задания на произведение векторов, системы линейных уравнений, операции с матрицами и проверку продуктивности матриц.

1.4 Объект и предмет исследования

Объект: объекты линейной алгебры (векторы, матрицы) и операции над ними.

Предмет: эффективность и корректность реализации линейно-алгебраических операций в пакетах Julia (`LinearAlgebra`, `Statistics`, `BenchmarkTools`).

1.5 Методы исследования

- численные методы линейной алгебры (LU-, QR-, спектральное разложение);
- проверка продуктивности матриц по трём критериям (по определению, через обратную матрицу, спектральный);
- бенчмаркинг производительности с помощью `@btime`.

1.6 Схема выполнения работы

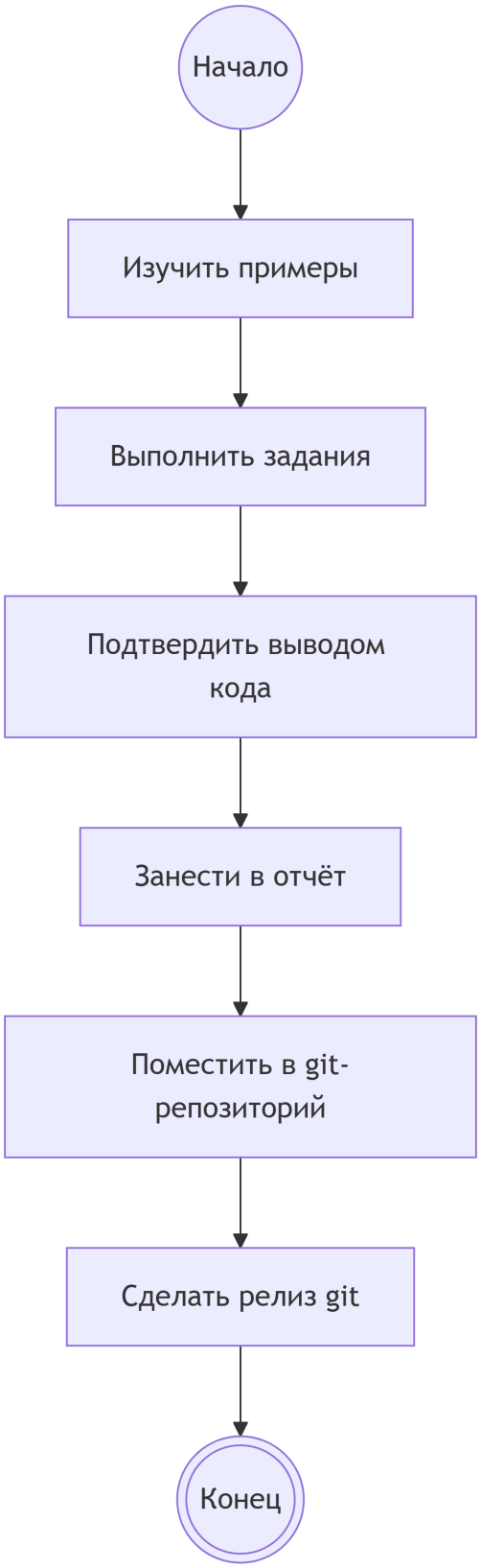


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Позлементные операции. Для массива a функции `sum`, `prod`, `mean` с аргументом `dims` позволяют агрегировать значения по столбцам (`dims=1`) или по строкам (`dims=2`).

Основные операции над матрицами. Транспонирование (`transpose`), след (`tr`), диагональные элементы (`diag`), ранг (`rank`), обратная матрица (`inv`), определитель (`det`), псевдообратная (`pinv`).

Нормы. Евклидова норма $\|\vec{X}\|_2 = \sqrt{\sum x_i^2}$, p-норма $\|\vec{X}\|_p = (\sum |x_i|^p)^{1/p}$. Угол между векторами: $\cos\theta = \dfrac{\vec{X}^T\vec{Y}}{\|\vec{X}\|_2\|\vec{Y}\|_2}$.

Факторизации. - LU: $A = LU$, где L — нижнетреугольная, U — верхнетреугольная. - LUP: $PA = LU$, P — матрица перестановок. - QR: $A = QR$, Q — ортогональная, R — верхнетреугольная. - Спектральное разложение: $A = V\Lambda V^{-1}$.

Продуктивность матрицы. Матрица A называется продуктивной, если решение x системы $x - Ax = y$ при любом неотрицательном y имеет только неотрицательные элементы. Эквивалентные критерии: 1. все элементы $(E - A)^{-1}$ неотрицательны; 2. все собственные значения A по модулю меньше 1.

3 Материалы и методы

- **Оборудование:** ПК.
- **Программное обеспечение:** Julia 1.12.7, пакеты `LinearAlgebra`, `Statistics`, `BenchmarkTools`; Jupyter Notebook.
- **Методы:** численные методы линейной алгебры, бенчмаркинг.

4 Выполнение лабораторной работы

4.1 Примеры

4.1.1 Позлементные операции над многомерными массивами

```
a = rand(1:20,(4,3))
sum(a); sum(a,dims=1); sum(a,dims=2)
prod(a); prod(a,dims=1); prod(a,dims=2)

import Pkg
Pkg.add("Statistics")
using Statistics
mean(a); mean(a,dims=1); mean(a,dims=2)
```

Результат:

```
4×1 Matrix{Float64}:
  9.0
 14.0
 14.333333333333334
 11.0
```

4.1.2 Транспонирование, след, ранг, определитель и инверсия матрицы

```
import Pkg
Pkg.add("LinearAlgebra")
using LinearAlgebra

b = rand(1:20,(4,4))
transpose(b); tr(b); diag(b); rank(b); inv(b); det(b); pinv(a)
```

4.1.3 Нормы векторов и матриц, повороты, вращения

```
X = [2, 4, -5]
norm(X)
p = 1; norm(X,p)

X = [2, 4, -5]; Y = [1, -1, 3];
norm(X- Y)
sqrt(sum((X-Y).^2))
acos((transpose(X)*Y)/(norm(X)*norm(Y)))

d = [5 -4 2 ; -1 2 3; -2 1 0]
opnorm(d)
p=1; opnorm(d,p)
rot180(d)
reverse(d,dims=1)
reverse(d,dims=2)
```

4.1.4 Матричное умножение, единичная матрица, скалярное произведение

```
A = rand(1:10,(2,3))
B = rand(1:10,(3,4))
A*B
Matrix{Int}(I, 3, 3)

X = [2, 4, -5]
Y = [1, -1,3]
dot(X,Y)
X'Y
```

4.1.5 Факторизация

```
A = rand(3, 3)
x = fill(1.0, 3)
b = A*x
A\b

Alu = lu(A)
Alu.P; Alu.p; Alu.L; Alu.U
Alu\b
det(A); det(Alu)

Aqr = qr(A)
Aqr.Q; Aqr.R
Aqr.Q' * Aqr.Q
```

4.1.6 Спектральное разложение и большие матрицы

```
Asym = A + A'
AsymEig = eigen(Asym)
AsymEig.values
AsymEig.vectors
inv(AsymEig)*Asym

n = 1000
A = randn(n,n)
Asym = A + A'
issymmetric(Asym)

Asym_noisy = copy(Asym)
Asym_noisy[1,2] += 5eps()
issymmetric(Asym_noisy)

Asym_explicit = Symmetric(Asym_noisy)

import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools

@btime eigvals(Asym);
@btime eigvals(Asym_noisy);
@btime eigvals(Asym_explicit);

n = 1000000;
A = SymTridiagonal(randn(n), randn(n-1))
```

```
@btime eigmax(A)
```

Результаты бенчмаркинга:

```
150.611 ms (18 allocations: 7.98 MiB)     # eigvals(Asym)
  1.421 s (27 allocations: 7.93 MiB)    # eigvals(Asym_noisy)
148.155 ms (18 allocations: 7.98 MiB)    # eigvals(Asym_explicit)
  1.124 s (39 allocations: 183.11 MiB)    # eigmax(A)
```

Вывод: явное указание симметричности (`Symmetric`) возвращает эффективность к уровню симметричной матрицы, тогда как зашумлённая матрица обрабатывается как общего вида (в ~10 раз медленнее).

4.2 Задания

4.2.1 Задание 1. Произведение векторов

```
v = [1, 2, 3]
dot_v = dot(v,v)       # 14
outer_v = v * v'
```

Результат:

```
dot_v = 14
outer_v =
 1  2  3
 2  4  6
 3  6  9
```

4.2.2 Задание 2. Системы линейных уравнений

Решение СЛАУ с двумя и тремя неизвестными. Для совместных систем использовалась операция \, для недоопределённых — `pinv(A)*b`, для анализа — ранги матриц.

Основные результаты:

```
1a: x = [2.5, -0.5]                          # [illegible]
1b: rank(A)=1, rank([A b])=1 -> [illegible]
1c: rank(A)=1, rank([A b])=2 -> [illegible]
1d: rank(A)=1, rank([A b])=1 -> [illegible]
1e: rank(A)=2, rank([A b])=3 -> [illegible]
1f: rank(A)=2, rank([A b])=2 -> x = [-1.0, 3.0]
2a: rank(A)=2, rank([A b])=2 -> [illegible] x = [2.214, 0.357, -0.571]
2b: det(A)=-10.0 -> x = [-0.5, 2.5, 0.0]
2c: rank(A)=2, rank([A b])=2 -> [illegible] x = [1.0, 1.0, -1.0]
2d: rank(A)=2, rank([A b])=3 -> [illegible]
```

4.2.3 Задание 3. Операции с матрицами

1. Приведение матриц к диагональному виду:

```
A = [1 -2; -2 1]
E = eigen(A)
V = E.vectors
Λ = Diagonal(E.values)
A_diag = V * Λ * inv(V)
```

Результаты:

```
1a: □ = [-1.0, 3.0]
1b: □ = [2.0, 4.0]
1c: □ = [-1.0, 3.0]
```

2. Вычисление:

```
A = [1 -2; -2 1]
A^10
B = [5 -2; -2 5]; sqrt(B)
A^(1/3)
C = [1 2; 2 3]; sqrt(C)
```

Результаты:

```
2a: A^10 = [29525 -29524; -29524 29525]
2b: sqrt(B) = [2.1889 -0.45685; -0.45685 2.1889]
2c: A^(1/3) — □□□□□□□□□□□ □□□□□□□
2d: sqrt(C) — □□□□□□□□□□□ □□□□□□□
```

3. Собственные значения матрицы A (5×5):

```
A = [140  97  74 168 131;
      97 106  89 131  36;
      74  89 152 144  71;
     168 131 144  54 142;
     131  36  71 142  36]
E = eigen(A)
```

Результат:

```
□ = [-128.493, -55.888, 42.752, 87.161, 542.468]
□□□□□□□ □□□□□□□□□□□? true
```

4.2.4 Задание 4. Линейная модель экономики

Проверка продуктивности матриц тремя критериями.

1. По определению (`x - Ax = y`):

```
1a: A = [1 2; 3 4]          -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
1b: A = 0.5·[1 2; 3 4]      -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
1c: A = 0.1·[1 2; 3 4]      -> ▒▒▒▒▒▒▒▒▒▒▒▒▒
```

2. Через $(E - A)^{-1} \geq 0$**:**

```
2a: A = [1 2; 3 1]          -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
2b: A = 0.5·[1 2; 3 1]      -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
2c: A = 0.1·[1 2; 3 1]      -> ▒▒▒▒▒▒▒▒▒▒▒▒▒
```

3. Спектральный критерий $\max|\lambda| < 1$**:**

```
3a: max|▒| = 3.449  -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
3b: max|▒| = 1.725  -> ▒▒ ▒▒▒▒▒▒▒▒▒▒▒▒
3c: max|▒| = 0.345  -> ▒▒▒▒▒▒▒▒▒▒▒▒▒
3d: max|▒| = 0.373  -> ▒▒▒▒▒▒▒▒▒▒▒▒▒
```

5 Результаты

1. Освоены базовые операции Julia для работы с массивами и матрицами.
2. Получены навыки работы с LU-, QR- и спектральным разложением.
3. Продемонстрирована эффективность специальных матричных структур (`Symmetric`, `SymTridiagonal`) при бенчмаркинге.
4. Решены 6 СЛАУ с двумя неизвестными и 4 СЛАУ с тремя неизвестными.
5. Найдены собственные значения матрицы 5×5.
6. Проверена продуктивность матриц тремя критериями — результаты согласованы.

6 Обсуждение результатов

- **Согласованность критериев продуктивности.** Все три критерия дали одинаковые результаты для каждой матрицы, что подтверждает корректность реализации и справедливость теорем.
- **Производительность.** Использование явной структуры `Symmetric` ускоряет вычисление собственных значений в ~10 раз по сравнению с матрицей общего вида. Трёхдиагональная структура `SymTridiagonal` позволяет находить максимальное собственное значение для матрицы размера $10^6 \times 10^6$ за ~1.1 с.
- **Комплексные результаты.** Для матриц, не являющихся положительно определёнными, `sqrt` и `A^(1/3)` дают комплексные матрицы — это математически корректно, но требует внимания при интерпретации.
- **Несовместные системы.** В пунктах 1c, 1e, 2d системы несовместны (rank(A) < rank([A b])), что подтверждено вычислением рангов.

7 Заключение

- **Цель достигнута:** изучены возможности пакетов Julia для выполнения операций линейной алгебры и оценки их эффективности.
- **Основные результаты:** освоены поэлементные операции, факторизации, спектральное разложение; решены все задания; подтверждена продуктивность матриц тремя критериями.
- **Рекомендации:** при работе с большими матрицами следует явно указывать их структуру (`Symmetric`, `SymTridiagonal`, `Diagonal` и др.) для повышения производительности.

Список литературы

A Приложение А. Полный листинг программы

Полный код лабораторной работы приведён в прилагаемом файле `lab4.ipynb`.

В Приложение Б. Результаты бенчмаркинга

```
150.611 ms (18 allocations: 7.98 MiB)     # eigvals(Asym)
  1.421 s (27 allocations: 7.93 MiB)    # eigvals(Asym_noisy)
148.155 ms (18 allocations: 7.98 MiB)    # eigvals(Asym_explicit)
  1.124 s (39 allocations: 183.11 MiB)   # eigmax(A)
```